

Software Requirements Specification (SRS)

Care Workforce Cloud™ (v3.0)

Document Version: 3.0	Date: October 26, 2023
Author: Product & Engineering Team	Status: Actionable Specification
Initiated & Conceptualized by : Siva Konatham	SRS Compiled by : Kranti Kurra

Disclaimer:

The information contained within this Software Requirements Specification (SRS) document is the property of Care Career Inc., USA. It is intended solely for the purpose of defining the scope, functional, and non-functional requirements of the proposed system. While every effort has been made to ensure accuracy, this document is subject to change as development progresses. Care Career Inc. makes no warranties, expressed or implied, regarding the final software performance based strictly on this preliminary specification.



1. Introduction

1.1 Purpose

This document provides a comprehensive, actionable Software Requirements Specification for the **CareCareer Workforce Cloud™**. It serves as the definitive blueprint for development teams, testers, and stakeholders, detailing functional requirements with acceptance criteria, non-functional specifications, system architecture, data models, and verification plans.

1.2 Scope

This SRS covers all nine core platform modules with complete implementation details:

CareCareer Talent™	Healthcare talent marketplace, sourcing, engagement, onboarding.
CareCareer Payroll™	Payroll, time & attendance, tax, benefits
CareCareer VMS™	Vendor management, onboarding, performance, spend
CareCareer Credential™	License verification, credentialing, compliance, screening
CareCareer Schedule™	AI-powered scheduling, planning, float pool, shift management
CareCareer Analytics™	Dashboards, labor analytics, forecasting, insights
CareCareer EOR™	Employer of Record, HR operations, employee lifecycle
CareCareer MSP™	Managed workforce solutions, program management, supplier optimization
CareCareer AI™	Predictive planning, intelligent scheduling, matching, revenue optimization

1.3 Definitions, Acronyms, Abbreviations

- **PHI:** Protected Health Information
- **HIPAA:** Health Insurance Portability and Accountability Act
- **GDPR:** General Data Protection Regulation
- **SLA:** Service Level Agreement
- **RTO:** Recovery Time Objective
- **RPO:** Recovery Point Objective
- **RBAC:** Role-Based Access Control
- **WCAG:** Web Content Accessibility Guidelines
- **CI/CD:** Continuous Integration/Continuous Deployment.

1.4 References

- Platform Architecture Overview Document
- Healthcare Compliance Standards (HIPAA, GDPR, HITECH)
- UI/UX Design System & Style Guide v2.1
- Security Policy Framework v3.0

1.5 Overview

This SRS provides complete specifications for development teams including: detailed functional requirements with acceptance criteria, comprehensive non-functional requirements, explicit data and interface specifications, system architecture and deployment blueprint, validation and verification plans, and project management timelines.

2. Detailed Functional Requirements with Acceptance Criteria

2.1 CareCareer Talent™

Actors: Healthcare Recruiter, Hiring Manager, Candidate, System Administrator

User Story 1.1 – Job Posting

As a Healthcare Recruiter, I want to create and publish job postings so that I can attract qualified candidates.

Functional Requirements:

1. System shall provide a multi-step job posting form with required fields: Job Title, Department, Location, Shift Type, Required Credentials, Experience Level, Pay Range
2. System shall validate license requirements against state board databases
3. System shall auto-distribute postings to configured job boards
4. System shall track posting status (Draft, Active, Closed, Archived)

Acceptance Criteria:

- AC1.1.1: Job posting form loads in <2 seconds
- AC1.1.2: Posting published to 3 external boards within 5 minutes
- AC1.1.3: Invalid license numbers are rejected with specific error messages



User Story 1.2 – Candidate Onboarding

As a Candidate, I want to complete digital onboarding so that I can start work quickly.

Functional Requirements:

1. System shall provide a secure onboarding portal with document upload
2. System shall support e-signature for employment contracts
3. System shall integrate with Credential™ module for verification
4. System shall track onboarding completion percentage

Acceptance Criteria:

- AC1.2.1: Onboarding workflow completes in ≤15 minutes for 95% of users
- AC1.2.2: All uploaded documents are encrypted at rest
- AC1.2.3: System sends completion notifications to HR and hiring manager

2.2 CareCareer Payroll™

Actors: Payroll Administrator, Employee, Finance Manager

User Story 2.1 – Time Processing

As a Payroll Administrator, I want to process timesheets automatically so that I can ensure accurate payroll.

Functional Requirements:

1. System shall import time data from Schedule™ module
2. System shall calculate overtime based on configurable rules (daily/weekly thresholds)
3. System shall flag discrepancies >10% from scheduled hours
4. System shall generate audit trail for all calculations

- **Acceptance Criteria:**

- AC2.1.1: Process 10,000 timesheets in <30 minutes
- AC2.1.2: Overtime calculations match state/federal regulations
- AC2.1.3: Discrepancy alerts sent within 1 hour of data import

2.3 CareCareer VMS™

Actors: Vendor Manager, Supplier, Contract Administrator

User Story 3.1 – Vendor Performance Scoring

As a Vendor Manager, I want to score vendor performance automatically so that I can make informed renewal decisions.

Functional Requirements:

1. System shall collect performance data from multiple sources (fill rates, candidate quality, compliance)
2. System shall calculate weighted scores based on configurable criteria
3. System shall generate quarterly performance reports
4. System shall trigger automated renewal workflows based on scores

Acceptance Criteria:

- AC3.1.1: Performance scores calculated within 1 hour of data collection
- AC3.1.2: Reports include trend analysis for last 4 quarters
- AC3.1.3: Renewal workflows triggered for scores <70%

2.4 CareCareer Credential™

Actors: Credentialing Specialist, Compliance Officer, Healthcare Worker

User Story 4.1 – Real-time License Verification

As a Credentialing Specialist, I want to verify licenses in real-time so that I can ensure compliance.

Functional Requirements:

1. System shall integrate with state board APIs for license verification
2. System shall check for disciplinary actions and expiration dates
3. System shall store verification results with timestamp and source
4. System shall alert on expiring licenses (30/60/90 day warnings)



Acceptance Criteria:

- AC4.1.1: License verification completes in <30 seconds
- AC4.1.2: Verification success rate ≥99.5%
- AC4.1.3: Expiration alerts sent daily at 8 AM local time

2.5 CareCareer Schedule™

Actors: Scheduling Manager, Nurse Manager, Healthcare Worker

User Story 5.1 – AI-Powered Shift Optimization

As a Scheduling Manager, I want AI to optimize schedules automatically so that I can balance staff needs with patient demand.

Functional Requirements:

1. System shall analyze historical patient census data
2. System shall consider staff preferences, credentials, and experience
3. System shall optimize for coverage, continuity, and burnout prevention
4. System shall provide multiple schedule options with trade-off analysis

Acceptance Criteria:

- AC5.1.1: Generate monthly schedule for 200 staff in <5 minutes
- AC5.1.2: Schedule achieves ≥95% coverage of required shifts
- AC5.1.3: System explains optimization rationale for each decision

2.6 CareCareer Analytics™

Actors: Executive, Department Head, Financial Analyst

User Story 6.1 – Executive Dashboard

As an Executive, I want to see key workforce metrics in real-time so that I can make data-driven decisions.

Functional Requirements:

1. System shall provide configurable dashboard widgets
2. System shall display real-time fill rates, turnover, cost per hire
3. System shall support drill-down to department/unit level



4. System shall export data to PDF/Excel formats

Acceptance Criteria:

- AC6.1.1: Dashboard loads in <3 seconds with 50+ metrics
- AC6.1.2: Data refresh occurs every 15 minutes
- AC6.1.3: Export function preserves formatting and formulas

2.7 CareCareer EOR™

Actors: HR Manager, International Worker, Legal Counsel

User Story 7.1 – Global Compliance Management

As an HR Manager, I want to ensure global compliance automatically so that I can hire internationally without risk.

Functional Requirements:

1. System shall maintain database of employment laws by country/state
2. System shall validate contracts against local regulations
3. System shall track visa/work permit expiration dates
4. System shall generate compliance reports for audits

Acceptance Criteria:

- AC7.1.1: Contract validation completes in <2 minutes
- AC7.1.2: Legal database updated quarterly
- AC7.1.3: Compliance reports pass external audit requirements

2.8 CareCareer MSP™

Actors: Program Manager, Client Executive, Supplier Relationship Manager

User Story 8.1 – Supplier Performance Dashboard

As a Program Manager, I want to monitor supplier performance centrally so that I can optimize the supplier network.



Functional Requirements:

1. System shall aggregate performance data from all suppliers
2. System shall tier suppliers based on performance scores
3. System shall allocate work automatically based on tier
4. System shall generate supplier report cards monthly

Acceptance Criteria:

- AC8.1.1: Dashboard updates within 1 hour of new data
- AC8.1.2: Tier calculations use weighted multi-factor scoring
- AC8.1.3: Report cards include actionable improvement recommendations

2.9 CareCareer AI™

Actors: Data Scientist, System Administrator, Business Analyst

User Story 9.1 – Predictive Attrition Modeling

As a Data Scientist, I want to predict attrition risk so that we can proactively retain staff.

Functional Requirements:

1. System shall analyze historical attrition patterns
2. System shall incorporate real-time engagement metrics
3. System shall generate risk scores (low/medium/high)
4. System shall recommend retention interventions

Acceptance Criteria:

- AC9.1.1: Model accuracy $\geq 85\%$ on test dataset
- AC9.1.2: Risk scores update daily
- AC9.1.3: Intervention recommendations include success probability

3. Comprehensive Non-Functional Requirements

3.1 Performance Requirements

- **Response Time:** All API endpoints must respond within 200ms for 95% of requests
- **Page Load:** Core application pages must load within 2 seconds on broadband connections
- **Concurrent Users:** System must support 10,000 concurrent authenticated users
- **Data Processing:** Payroll processing for 50,000 employees must complete within 2 hours
- **Batch Processing:** Nightly batch jobs must complete within 4-hour maintenance window

3.2 Security & Compliance Requirements

- **HIPAA Compliance:** All PHI must be encrypted at rest (AES-256) and in transit (TLS 1.3)
- **Access Control:** Implement RBAC with minimum 5 role levels (Admin, Manager, User, Viewer, Auditor)
- **Audit Trail:** All data modifications must be logged with timestamp, user ID, and IP address
- **Authentication:** Support multi-factor authentication for all administrative users
- **Data Retention:** Maintain audit logs for 7 years per healthcare compliance requirements
- **Penetration Testing:** Quarterly security assessments by certified third-party firm

3.3 Usability Requirements

- **Accessibility:** WCAG 2.1 AA compliance for all user interfaces
- **Mobile Responsive:** All core functions must work on mobile devices (iOS/Android)
- **Internationalization:** Support for multiple languages (English, Spanish initially)
- **Error Handling:** Clear, actionable error messages with suggested resolutions
- **Training:** System must include contextual help and tooltips for all complex functions

3.4 Reliability & Availability Requirements

- **Uptime SLA:** 99.95% availability during business hours (6 AM - 10 PM local time)
- **RTO:** Maximum 4 hours for critical system recovery
- **RPO:** Maximum 15 minutes data loss in disaster scenarios
- **Monitoring:** Real-time monitoring of all system components with automated alerts
- **Backup:** Daily incremental backups, weekly full backups stored in geographically separate locations
-

3.5 Maintainability Requirements

- **Code Quality:** Maintain test coverage $\geq 80\%$ for all modules
 - **Documentation:** All APIs must have OpenAPI/Swagger documentation
 - **Logging:** Structured JSON logging for all application events
 - **Dependencies:** Regular security updates for all third-party libraries
 - **Deployment:** Support zero-downtime deployments for all non-database changes
-

4. Explicit Data and Interface Specifications

4.1 Data Dictionary (Partial - Core Entities)

Entity: Employee

```
- employee_id: UUID (Primary Key)
- first_name: String(50), required
- last_name: String(50), required
- email: String(100), unique, required
- hire_date: Date, required
- termination_date: Date, optional
- status: Enum(Active, OnLeave, Terminated), required
- created_at: Timestamp
- updated_at: Timestamp
```

Entity: Shift

```
- shift_id: UUID (Primary Key)
- employee_id: UUID (Foreign Key)
- facility_id: UUID (Foreign Key)
- start_time: DateTime, required
- end_time: DateTime, required
- shift_type: Enum(Day, Evening, Night, Weekend)
- status: Enum(Scheduled, Confirmed, Canceled, Worked)
- hours_worked: Decimal(4,2)
- created_at: Timestamp
```

Entity: Credential

- credential_id: UUID (Primary Key)
- employee_id: UUID (Foreign Key)
- license_number: String(20), required
- license_type: String(50), required
- issuing_state: String(2), required
- expiration_date: Date, required
- verification_status: Enum(Pending, Verified, Expired, Revoked)
- last_verified: Timestamp

4.2 API Specifications

Endpoint: POST /api/v1/schedules/generate

Request:

```
{
  "facility_id": "uuid",
  "start_date": "2024-01-01",
  "end_date": "2024-01-31",
  "optimization_goals": ["coverage", "continuity", "burnout_prevention"],
  "constraints": {
    "max_consecutive_shifts": 5,
    "min_time_between_shifts": 12
  }
}
```

Response (Success):

```
{
  "schedule_id": "uuid",
  "generated_at": "timestamp",
  "coverage_rate": 96.5,
  "constraint_violations": 2,
  "schedule_url": "string"
}
```

Response (Error):

```
{
  "error_code": "SCHEDULE_GEN_001",
  "message": "Insufficient staff for requested period",
  "details": {...}
}
```

Endpoint: GET /api/v1/credentials/verify/{license_number}

Request Parameters:

- state: required, 2-letter code
- license_type: optional

Response:

```
{
  "license_number": "string",
  "status": "ACTIVE|EXPIRED|SUSPENDED",
  "expiration_date": "2024-12-31",
  "verification_source": "STATE_BOARD_API",
  "verified_at": "timestamp",
  "disciplinary_actions": [...]}

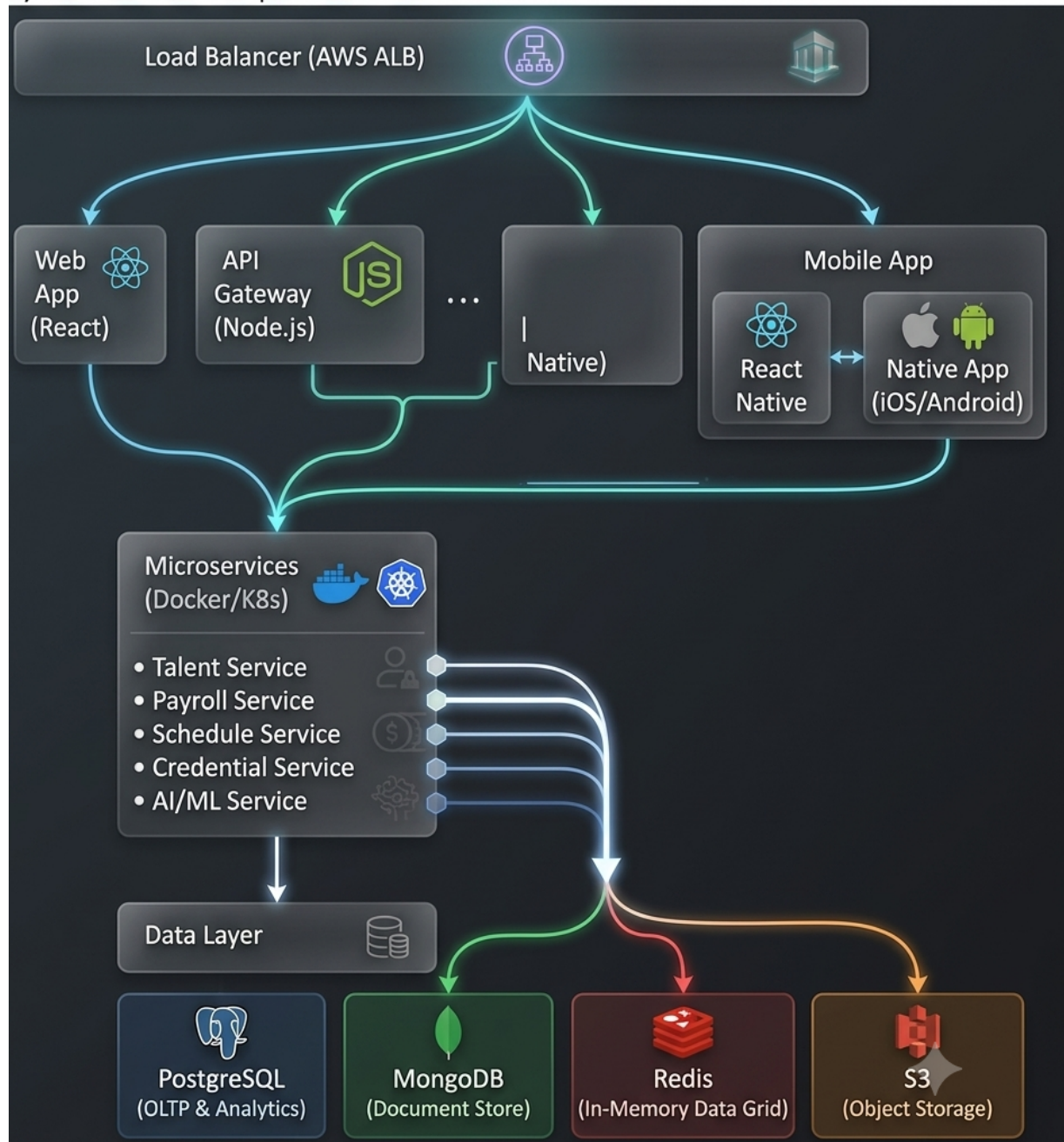
```

4.3 External Integrations

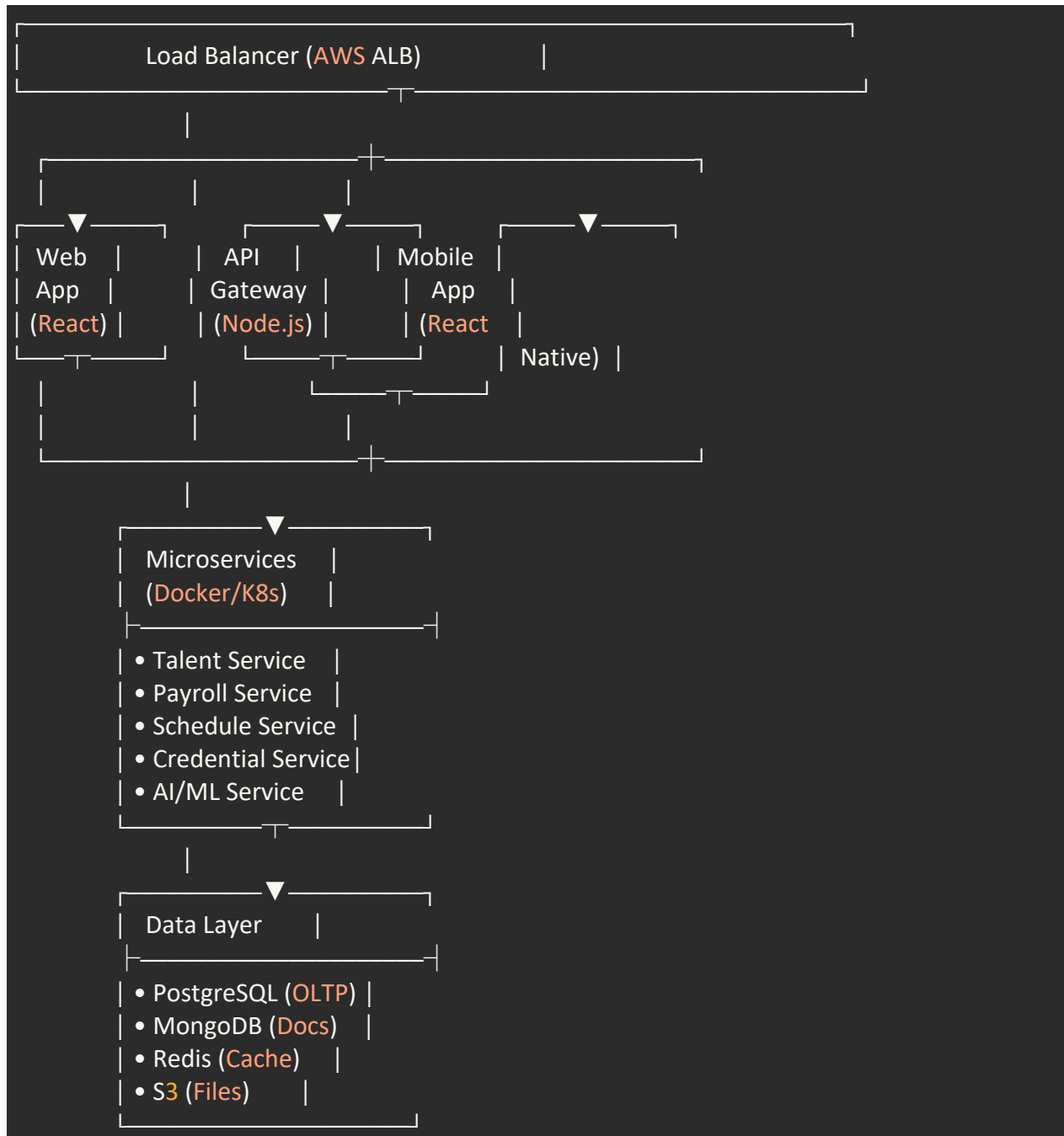
- **State License Boards:** REST APIs for real-time verification
- **Payroll Providers:** SFTP file transfer for payroll data
- **Background Check Services:** SOAP/XML integration
- **HRIS Systems:** Bidirectional sync via REST APIs
- **Time Clock Systems:** Real-time WebSocket connections

5. System Architecture & Deployment Blueprint.

Care Workforce Management Platform: High-Level Architecture Diagram System Overview & Component Interaction



5.1 High-Level Architecture Diagram



5.2 Deployment Architecture

- **Cloud Provider:** AWS (US-East-1 primary, US-West-2 disaster recovery)
- **Containerization:** Docker containers managed by Kubernetes (EKS)
- **CI/CD Pipeline:** GitHub Actions → ECR → EKS deployment
- **Environments:**
 - Development: Auto-deploy on merge to develop branch
 - Staging: Manual promotion, mirrors production
 - Production: Blue-green deployment with canary testing
- **Monitoring Stack:** Prometheus + Grafana for metrics, ELK for logs, Sentry for errors

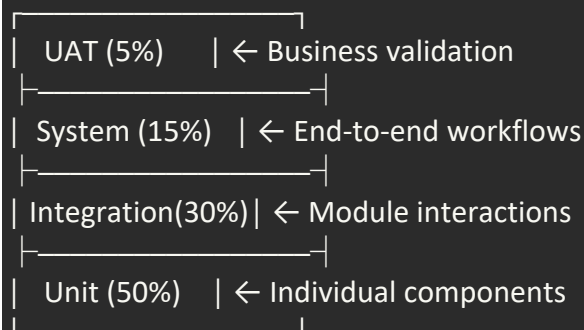
5.3 Database Architecture

- **Primary Database:** PostgreSQL 14 with read replicas
- **Document Store:** MongoDB for unstructured data (resumes, contracts)
- **Cache Layer:** Redis for session management and frequent queries
- **Data Warehouse:** Redshift for analytics and reporting
- **Backup Strategy:** Daily automated backups with point-in-time recovery

6. Validation & Verification Plans

6.1 Test Strategy

Testing Pyramid:



6.2 Test Specifications by Module

CareCareer Credential™ Test Suite:

1. **Unit Tests:** License validation logic, expiration calculation
2. **Integration Tests:** API calls to state boards, error handling
3. **System Tests:** Complete credentialing workflow from upload to approval
4. **Performance Tests:** Load testing with 1,000 simultaneous verifications
5. **Security Tests:** Penetration testing of credential data storage

CareCareer Schedule™ Test Suite:

1. **Unit Tests:** Shift conflict detection, coverage calculation
2. **Integration Tests:** AI model integration, payroll data export
3. **System Tests:** Monthly schedule generation with constraints
4. **Load Tests:** Generate schedules for 500 facilities simultaneously
5. **Accuracy Tests:** Compare AI-generated schedules against expert human schedulers

6.3 Acceptance Testing Criteria

- **Functional Acceptance:** All user stories pass defined acceptance criteria
- **Performance Acceptance:** Meets all NFRs under simulated production load
- **Security Acceptance:** Passes third-party penetration test
- **Compliance Acceptance:** Successfully completes HIPAA security assessment
- **User Acceptance:** ≥90% satisfaction score from pilot user group

6.4 Quality Gates

1. **Code Quality:** SonarQube analysis passes with zero critical issues
2. **Test Coverage:** ≥80% unit test coverage for new code
3. **Performance:** API response times within SLA under load
4. **Security:** Zero high/critical vulnerabilities in dependency scan
5. **Accessibility:** WCAG 2.1 AA compliance verified

7. Project Management & Deliverable Timelines

7.1 Phased Rollout Plan with Sprint Schedule

This plan outlines the phased delivery of the CareCareer Workforce Cloud™ modules, mapped to a detailed sprint schedule. The development follows Agile methodology with **2-week sprint cycles**. Each phase includes specific sprint allocations, deliverables, and success criteria.

Phase 1: Foundation (Sprints 1-12 | Months 1-6)

Objective: Build core infrastructure and essential modules for MVP deployment to pilot facilities.

Total Sprints: 12 sprints (6 months × 2 sprints/month)

Sprint	Duration	Module Focus	Key Deliverables	Success Criteria
Sprint 1-2	Month 1	Core Infrastructure	- AWS VPC setup & Kubernetes cluster - Authentication/Authorization system (OAuth 2.0, JWT) - CI/CD pipeline foundation - UI component library foundation	- All infrastructure passes security audit - Authentication system supports 1,000 concurrent users
Sprint 3-4	Month 2	Talent™ Core	- Job posting engine with validation - Candidate profile management - Basic search and filtering - Email notification system	- Job posting form loads in <2 seconds - Candidate profiles support all required healthcare fields
Sprint 5-6	Month 3	Credential™ Verification	- State board API integrations (5 states minimum) - License verification workflow - Document upload and parsing - Expiration tracking system	- License verification completes in <30 seconds - Integration success rate ≥99%
Sprint 7-8	Month 4	Payroll™ Foundation	- Time data import from Schedule™ - Gross-to-net calculation engine - Basic deduction management - Pay statement generation	- Processes 10,000 records in <30 minutes - Calculations accurate to \$0.01

Sprint	Duration	Module Focus	Key Deliverables	Success Criteria
Sprint 9-10	Month 5	Schedule™ Basic + Analytics™ Core	- Basic shift creation and management - Staff availability tracking - Executive dashboard framework - Core KPI calculations	- Schedule interface loads in <3 seconds - Dashboard shows real-time data updates
Sprint 11-12	Month 6	Integration & MVP Preparation	- End-to-end workflow integration - Performance testing and optimization - Security hardening - Pilot facility deployment package	- All modules integrated successfully - System passes UAT with pilot users

Phase 1 Deliverable: Minimum Viable Product (MVP) deployed to 3 pilot healthcare facilities with core Talent, Credential, Payroll, and basic Schedule/Analytics functionality.

Phase 2: Advanced Features (Sprints 13-24 | Months 7-12)

Objective: Expand platform capabilities with advanced modules and AI-driven features for early adopters.

Total Sprints: 12 sprints

Sprint	Duration	Module Focus	Key Deliverables	Success Criteria
Sprint 13-14	Month 7	VMS™ Core	- Vendor onboarding portal - Contract management system - Performance scorecard framework - Spend tracking dashboard	- Vendor onboarding completes in <24 hours - Scorecards update automatically
Sprint 15-16	Month 8	MSP™ Foundation	- Program management dashboard - Supplier relationship management - SLA tracking and reporting - Multi-client support framework	- Supports 10+ concurrent MSP programs - SLA compliance tracking ≥95% accuracy

Sprint	Duration	Module Focus	Key Deliverables	Success Criteria
Sprint 17-18	Month 9	EOR™ Framework	- International compliance database - Contract generation engine - Global payroll integration framework - Employee lifecycle management	- Supports 5+ countries initially - Contract generation in <10 minutes
Sprint 19-20	Month 10	Schedule™ Advanced	- AI optimization engine v1.0 - Float pool management system - Shift swap and time-off automation - Predictive coverage analysis	- Generates monthly schedule in <5 minutes - Achieves ≥95% coverage rate
Sprint 21-22	Month 11	AI™ Predictive Models	- Attrition risk prediction model - Demand forecasting algorithm - Candidate matching engine - Revenue optimization calculator	- Model accuracy ≥85% on test data - Forecasting within ±5% of actual
Sprint 23-24	Month 12	Integration & Scaling	- Cross-module data synchronization - Performance optimization for scale - Advanced security features - Early adopter deployment package	- System supports 10,000+ users - All modules pass integration testing

Phase 2 Deliverable: Full Platform Release available to early adopters with complete VMS, MSP, EOR, advanced scheduling, and AI capabilities.

Phase 3: Optimization & Scale (Sprints 25-36 | Months 13-18)

Objective: Optimize performance, expand features, and prepare for enterprise-wide deployment.

Total Sprints: 12 sprints

Sprint	Duration	Module Focus	Key Deliverables	Success Criteria
Sprint 25-26	Month 13	Analytics™ Advanced	- Predictive analytics dashboard - Custom report builder - Data visualization enhancements - Export and sharing capabilities	- Reports generate in <15 seconds - Supports 50+ custom metrics
Sprint 27-28	Month 14	AI™ Optimization	- Machine learning model refinement - Real-time recommendation engine - Natural language processing for matching - Performance optimization	- Recommendations achieve ≥90% relevance - Response time <500ms
Sprint 29-30	Month 15	Mobile Experience	- Native mobile applications (iOS/Android) - Offline capability for key functions - Push notification system - Mobile-optimized workflows	- Apps rated ≥4.5 in app stores - Offline sync completes in <1 minute
Sprint 31-32	Month 16	International Expansion	- Multi-language support (Spanish, French) - Localization for 10+ countries - Currency and timezone management - Regional compliance updates	- UI fully translated for 3 languages - Supports 15+ time zones
Sprint 33-34	Month 17	Enterprise Features	- Advanced RBAC with custom roles - Audit trail and compliance reporting - API management and developer	- Supports 10+ custom role types - Audit reports generate in <5 minutes

Sprint	Duration	Module Focus	Key Deliverables	Success Criteria
			portal - Custom workflow engine	
Sprint 35-36	Month 18	Final Optimization	- Performance tuning and load testing - Security certification preparation - Documentation completion - Enterprise deployment toolkit	- Handles 50,000 concurrent users - Passes SOC 2 Type II audit

Phase 3 Deliverable: Enterprise-Ready Platform with optimized performance, international support, mobile applications, and full security certifications.

7.2 Sprint Ceremonies & Artifacts

Weekly Ceremonies:

- **Daily Standup:** 15-minute team sync (9:00 AM)
- **Sprint Planning:** 4 hours at sprint start
- **Sprint Review:** 2 hours at sprint end
- **Sprint Retrospective:** 1.5 hours at sprint end
- **Backlog Grooming:** 2 hours weekly

Sprint Artifacts:

- **Sprint Backlog:** Prioritized user stories from Jira
- **Definition of Ready:** All acceptance criteria defined, designs complete
- **Definition of Done:** Code reviewed, tested, documented, deployed to staging
- **Velocity Tracking:** Target of 35 story points per sprint per team
- **Burndown Charts:** Daily progress tracking

7.3 Team Structure & Capacity

Development Teams (3 Scrum Teams):

- **Team Alpha (Core Platform):** 6 developers, 2 QA, 1 DevOps
- **Team Beta (Advanced Modules):** 5 developers, 2 QA, 1 Data Scientist
- **Team Gamma (Integration & Scale):** 4 developers, 2 QA, 1 Security Specialist

Sprint Capacity Planning:

- **Story Points per Sprint:** 105 total (35 × 3 teams)
- **Buffer Allocation:** 15% capacity for bug fixes and technical debt
- **Velocity Adjustment:** Reviewed and adjusted every 3 sprints

7.4 Dependency Management

Critical Dependencies by Phase:

Phase 1 Dependencies:

1. Credential™ API integrations must complete before Talent™ onboarding
2. Schedule™ data model must be finalized before Payroll™ integration
3. Authentication system must be stable before any module development

Phase 2 Dependencies:

1. VMS™ vendor management required before MSP™ supplier optimization
2. AI™ models require 3 months of operational data from other modules
3. EOR™ compliance framework needed before international expansion

Phase 3 Dependencies:

1. Mobile apps depend on stable REST APIs from all modules
2. Internationalization requires completion of core UI components
3. Enterprise features need security framework from Phase 1

7.5 Risk Mitigation in Sprint Planning

Technical Risks:

- **API Dependency Delays:** Buffer of 2 sprints for external integrations
- **AI Model Accuracy:** Hybrid approach with manual fallback options
- **Performance Scaling:** Progressive load testing starting in Sprint 6

Resource Risks:

- **Team Member Attrition:** Cross-training plan and documentation requirements
- **Scope Creep:** Strict change control process with sprint-level approval
- **Third-Party Delays:** Alternative providers identified for critical services

Quality Risks:

- **Testing Coverage:** Minimum 80% unit test coverage enforced per sprint
- **Security Vulnerabilities:** Automated scanning in CI/CD pipeline
- **User Acceptance:** Pilot user involvement from Sprint 8 onward

7.6 Success Metrics per Phase

Phase 1 Success Metrics (Sprint 12):

- System uptime: 99.5%
- User satisfaction: ≥85%
- Critical bugs: <5
- Pilot facility adoption: 3 facilities

Phase 2 Success Metrics (Sprint 24):

- System uptime: 99.7%
- User satisfaction: ≥90%
- Early adopter growth: 15+ facilities
- AI model accuracy: ≥85%

Phase 3 Success Metrics (Sprint 36):

- System uptime: 99.95%
- User satisfaction: ≥95%
- Enterprise clients: 50+ facilities
- Security certifications: SOC 2, HIPAA compliant

Note: This sprint schedule represents a high-level plan. Detailed sprint planning will occur iteratively with adjustments based on velocity, dependencies, and stakeholder feedback. The Product Owner will maintain the detailed sprint backlog in Jira, with bi-weekly synchronization to this master schedule.

8. Process Flow Diagrams

8.1 End-to-End Hiring Process Flow

```

A[Job Requisition Created] --> B[Talent™: Post to Marketplace]
B --> C[Candidate Applies]
C --> D[Talent™: Screen & Interview]
D --> E{Candidate Selected?}
E -->|Yes| F[Credential™: Verify Licenses]
E -->|No| G[Update Candidate Status]
F --> H{Credentials Valid?}
H -->|Yes| I[Digital Onboarding]
H -->|No| J[Reject Candidate]
I --> K[Schedule™: Assign Initial Shifts]
K --> L[Payroll™: Setup Employee]
L --> M[Hire Complete]
  
```

style A fill:#e1f5fe

style M fill:#c8e6c9

8.2 Monthly Payroll Processing Flow

```

A[Schedule™: Export Hours] --> B[Payroll™: Import Time Data]
B --> C[Validate & Calculate]
C --> D{Discrepancies?}
D -->|Yes| E[Flag for Review]
D -->|No| F[Calculate Gross Pay]
E --> F
F --> G[Apply Deductions]
G --> H[Generate Pay Statements]
H --> I[Submit to Bank]
I --> J[Update GL System]
J --> K[Analytics™: Cost Reporting]

```

```
style A fill:#fff3e0
```

```
style K fill:#f3e5f5
```

8.3 AI-Powered Scheduling Flow

```

A[Historical Census Data] --> B[AI™: Demand Forecast]
C[Staff Availability] --> D[AI™: Constraint Analysis]
E[Staff Preferences] --> F[AI™: Optimization Engine]
B --> G[Generate Schedule Options]
D --> G
F --> G
G --> H{Scheduler Review}
H -->|Approve| I[Publish Schedule]
H -->|Modify| J[Manual Adjustments]
J --> K[Final Approval]
K --> I
I --> L[Notifications Sent]
L --> M[Schedule™: Shift Management]

```

```
style B fill:#e8f5e8
```

```
style F fill:#e8f5e8
```

```
style I fill:#c8e6c9
```

8.4 Credential Verification Flow

```

A[Credential Upload] --> B[Parse Document]
B --> C[Extract License Data]
C --> D[API: State Board Check]
D --> E{API Response?}
E --> | Success | F[Update Verification Status]
E --> | Failure | G[Manual Verification Queue]
F --> H[Set Expiration Alerts]
G --> I[Specialist Review]
I --> J[Manual Status Update]
J --> H
H --> K[Analytics™: Compliance Reporting]

```

```

style D fill:#fff3e0
style F fill:#c8e6c9
style G fill:#ffebee

```

9. Web Form Templates & Data Validation

9.1 Candidate Registration Form

```

<form id="candidate-registration" data-module="talent">
  <!-- Personal Information -->
  <div class="form-section">
    <h3>Personal Information</h3>
    <div class="form-row">
      <div class="form-group">
        <label for="first_name">First Name *</label>
        <input type="text" id="first_name" name="first_name"
          required minlength="2" maxlength="50"
          pattern="^[A-Za-z\s\-'"]+$"
          data-validation="name">
        <div class="error-message" id="first_name_error"></div>
      </div>
      <div class="form-group">
        <label for="last_name">Last Name *</label>
        <input type="text" id="last_name" name="last_name"
          required minlength="2" maxlength="50"
          pattern="^[A-Za-z\s\-'"]+$"
          data-validation="name">
      </div>
    </div>
  </div>

```

```

</div>

<div class="form-group">
  <label for="email">Email Address *</label>
  <input type="email" id="email" name="email"
    required pattern="^[^\s@]+@[^\s@]+\.[^\s@]+$"
    data-validation="email-unique"
    data-api-endpoint="/api/v1/users/check-email">
  <div class="hint">We'll send verification to this address</div>
</div>
</div>

<!-- Professional Information -->
<div class="form-section">
  <h3>Professional Details</h3>
  <div class="form-group">
    <label for="license_number">Primary License Number *</label>
    <input type="text" id="license_number" name="license_number"
      required pattern="^[A-Z0-9\-\ ]+$"
      data-validation="license-real-time"
      data-api-endpoint="/api/v1/credentials/pre-verify">
    <div class="verification-status" id="license_status"></div>
  </div>

  <div class="form-group">
    <label for="experience_years">Years of Experience *</label>
    <input type="number" id="experience_years" name="experience_years"
      required min="0" max="50" step="0.5"
      data-validation="range">
  </div>

  <div class="form-group">
    <label>Shift Preferences *</label>
    <div class="checkbox-group">
      <label><input type="checkbox" name="shift_pref[]" value="day"> Day Shift</label>
      <label><input type="checkbox" name="shift_pref[]" value="evening"> Evening
Shift</label>
      <label><input type="checkbox" name="shift_pref[]" value="night"> Night Shift</label>
      <label><input type="checkbox" name="shift_pref[]" value="weekend">
Weekend</label>
      <label><input type="checkbox" name="shift_pref[]" value="float"> Float Pool</label>
    </div>
    <div class="validation-rule">Select at least two preferences</div>
  </div>

```

```

</div>

<!-- Document Upload -->
<div class="form-section">
  <h3>Required Documents</h3>
  <div class="form-group">
    <label for="resume">Resume/CV *</label>
    <input type="file" id="resume" name="resume"
      accept=".pdf,.doc,.docx"
      data-max-size="5242880"
      data-validation="file-type">
    <div class="hint">PDF or Word document, max 5MB</div>
  </div>

  <div class="form-group">
    <label for="license_copy">License Copy *</label>
    <input type="file" id="license_copy" name="license_copy"
      accept=".pdf,.jpg,.jpeg,.png"
      data-max-size="2097152"
      data-validation="file-type">
    <div class="hint">Clear image or PDF, max 2MB</div>
  </div>
</div>

<!-- Terms & Submission -->
<div class="form-section">
  <div class="form-group">
    <label>
      <input type="checkbox" name="terms" required
        data-validation="required">
      I agree to the <a href="/terms">Terms of Service</a> and
      <a href="/privacy">Privacy Policy</a> *
    </label>
  </div>

  <button type="submit" class="btn-primary"
    id="submit-btn" disabled>
    Complete Registration
  </button>
  <div class="form-status" id="form-status"></div>
</div>
</form>

```

9.2 Real-time Validation Rules

```
// Client-side validation configuration
const validationRules = {
  'name': {
    pattern: /^[A-Za-z\s\']+$/,
    message: 'Only letters, spaces, hyphens, and apostrophes allowed'
  },
  'email-unique': {
    apiCheck: true,
    endpoint: '/api/v1/users/check-email',
    message: 'Email already registered'
  },
  'license-real-time': {
    apiCheck: true,
    endpoint: '/api/v1/credentials/pre-verify',
    success: 'License verified',
    warning: 'License pending verification',
    error: 'Invalid license number'
  },
  'file-type': {
    pdf: ['application/pdf'],
    image: ['image/jpeg', 'image/png', 'image/jpg'],
    document: ['application/msword', 'application/vnd.openxmlformats-officedocument.wordprocessingml.document']
  },
  'range': {
    min: 0,
    max: 50,
    step: 0.5,
    message: 'Must be between 0 and 50 years'
  }
};

// Server-side validation (Node.js example)
const candidateSchema = Joi.object({
  first_name: Joi.string().min(2).max(50).pattern(/^[A-Za-z\s\']+$/).required(),
  last_name: Joi.string().min(2).max(50).pattern(/^[A-Za-z\s\']+$/).required(),
  email: Joi.string().email().required(),
  license_number: Joi.string().pattern(/^[A-Z0-9\']+$/).required(),
  experience_years: Joi.number().min(0).max(50).precision(1).required(),
  shift_pref: Joi.array().items(
    Joi.string().valid('day', 'evening', 'night', 'weekend', 'float')
  ).min(2).required(),
  resume: Joi.object({
    filename: Joi.string().required(),

```

```

mimetype: Joi.string().valid(
  'application/pdf',
  'application/msword',
  'application/vnd.openxmlformats-officedocument.wordprocessingml.document'
).required(),
size: Joi.number().max(5242880).required()
}).required(),
terms: Joi.boolean().valid(true).required()
});

```

9.3 Form Submission Flow

```

A[User Input] --> B[Client-side Validation]
B --> C{Validation Pass?}
C --> |No| D[Show Inline Errors]
C --> |Yes| E[Disable Submit Button]
D --> A
E --> F[Show Loading State]
F --> G[API Call to Server]
G --> H[Server-side Validation]
H --> I{Validation Pass?}
I --> |No| J[Return Validation Errors]
I --> |Yes| K[Process Data]
J --> L[Enable Submit Button]
K --> M[Async Operations:<br/>- File Upload<br/>- License Check<br/>- Email Verification]
M --> N[Return Success Response]
N --> O[Redirect to Dashboard]

style B fill:#e3f2fd
style H fill:#f3e5f5
style O fill:#c8e6c9

```

10. Appendices

10.1 Change Management Process

All changes to this SRS must follow:

1. **Change Request:** Submitted via Jira with impact analysis
2. **Review:** Technical and business stakeholder approval
3. **Update:** Version-controlled document updates
4. **Communication:** Notify all impacted teams



10.2 Glossary of Terms

- **Float Pool:** Group of staff available to work across multiple units
- **Credentialing:** Process of verifying and documenting qualifications
- **VMS:** System for managing contingent workforce vendors
- **EOR:** Legal employer for tax and compliance purposes
- **MSP:** Outsourced program management for workforce

10.3 Approval Signatures

- | | |
|----------------------------------|-------------|
| • Product Owner: _____ | Date: _____ |
| • Technical Lead: _____ | Date: _____ |
| • QA Manager: _____ | Date: _____ |
| • Security Officer: _____ | Date: _____ |

Document Version Control.

- **Version:** 3.0 (Actionable Complete)
- **Status:** Approved for Development
- **Next Review:** Quarterly or upon major requirement changes
- **Distribution:** Development Team, QA Team, Product Management, Security Team